

Aufgabe 1: Calculator: Anonyme Klassen und Lambda-Ausdrücke

Erstellen Sie eine neue Java-Klasse `Sub`, die das Interface `Operation` implementiert und eine Subtraktion bereitstellt. Erweitern Sie den `Calculator` und binden Sie eine Instanz dieser Klasse ein. Nutzen Sie hier keine anonymen Klassen oder Lambda-Ausdrücke.

Erstellen Sie eine weitere Operation `"Mul"` (Multiplikation von zwei Integern). Nutzen Sie dazu eine passende anonyme Klasse.

Erstellen Sie eine weitere Operation `"Div"` (Integerdivision). Erstellen Sie einen passenden Lambda-Ausdruck.

Für die `JComboBox operationSelector` wird ein `ActionListener` mit Hilfe einer anonymen Klasse definiert. Konvertieren Sie dies in einen entsprechenden Lambda-Ausdruck.

Neue Operation „Sub“ erstellt

- Es wurde eine neue Klasse `Sub` geschrieben, die das Interface `Operation` implementiert.
- Die Methode `doOperation(int a, int b)` führt eine Subtraktion aus.
- Die Klasse wurde im `Calculator` registriert:

Operation „Mul“ als anonyme Klasse hinzugefügt

- Es wurde eine anonyme Klasse direkt in der `HashMap` definiert

Operation „Div“ als Lambda-Ausdruck hinzugefügt

- Da `Operation` ein `Functional Interface` ist, konnte ein Lambda verwendet werden (Ein Lambda funktioniert, wenn das Interface ein **Functional Interface** ist. `ActionListener` hat genau **eine** abstrakte Methode)

ActionListener der JComboBox in Lambda umgewandelt

- Die ursprüngliche anonyme Klasse
- wurde ersetzt durch ein Lambda ersetzt

Alles wurde getestet und funktioniert.